

TERMS OF USE

=====

Document status: Active proof of concept and development state.

DevBox already has a functioning local launcher, a graphical main interface, modular structure views, master-data and documentation maintenance, documentation snapshots, local tool detection, a repository page, and initial building blocks for the DevBox-specific publication process.

The deskNode area is actively integrated into DevBox. Supervisor and daemon processes can be started and stopped from the GUI, and their console output is displayed in the deskNode log. The product version can be edited through master data. UX themes can be named, created, deleted, duplicated, renamed, and saved. Saving a theme refreshes the deskNode manufacturer database.

Symbol management now exists as an initial functional catalog interface. It manages deskNode device categories and consumer devices through selection menus and dialogs for creation, editing, and deletion. New devices receive one PNG source, stored under their record ID as `symbol_source_<record_id>.png`. After every successful catalog change, `create_manufacturer_db.py` is executed so `mnfctr_db.r0b` contains current theme, category, and device data.

The graphics-pack build is functional as a proof of concept. It creates multiple prepared consumer-graphics states from symbol sources and UX themes. Actual deskNode runtime integration, asset selection, language packages, and complete state logic are still being developed.

DevBox is not a finished end-user or release version. Interfaces, database migrations, product modules, publication workflows, document templates, and tests are continuously being revised. This documentation describes the current state and must be updated alongside functional changes.

First publication: Author / publisher: Markus Walloner Country: Germany (DE)

1. Subject Matter

These terms of use apply to the software or project described in the accompanying project documentation.

Short Description: DevBox is the local development hub of the CYXnTrol Development Platform. Its PySide6 interface brings together project structure, master data, documentation, local tools, product modules, repository preparation, and recurring development workflows.

DevBox is not an end-user product. It is an internal working environment in which technical requirements, data models, interfaces, build steps, and publication states are prepared, reviewed, and refined in a traceable manner.

Long Description: DevBox connects tasks that would otherwise be distributed across individual scripts, folders, spreadsheets, console windows, and external programs. It manages central project and product data, provides documentation content, organizes global structure templates, detects local tools, and brings product-specific development functions together in one interface.

An active integrated product module is deskNode. Its page can start and stop the local supervisor and daemon, edit the product version from master data, maintain UX themes, and open symbol management. Themes are maintained through named records, RGBA colors, global font files, sizes, outline settings, and shape rules. After each successful theme change, the product-local deskNode manufacturer database is refreshed.

Symbol management is the central source for device categories and consumer symbols. Categories and devices can be created, edited, or deleted in the DevBox database. When a new consumer device is created, exactly one PNG source is accepted and stored in the global graphics directory as `symbol_source_<record_id>.png`, based on the stable record ID. The graphics-pack build creates pre-rendered variants for themes and states so the later deskNode runtime only has to select suitable assets.

DevBox operates locally in the project context. A launcher discovers the project root through the `.root` file, provides runtime data under `AppData`, checks external tools, and starts the GUI from a temporary working copy. The real project root remains the authoritative source for resources, databases, and function scripts. A single-instance mechanism prevents parallel DevBox windows and activates an existing instance on a repeated start request.

The project is developed through an AI-assisted iterative workflow. Functional requirements, process logic, data models, UX decisions, test cases, and acceptance decisions are directed by the project owner; implementation steps are created with AI-assisted tools, reviewed, and continuously refined.

2. Terms of Use

TERMS OF USE – DRAFT VERSION

1. Scope

These terms of use apply to DevBox, the local development environment of the CYXnTrol Development Platform. DevBox is a development tool and not a finished productive end-user application.

2. Development status

DevBox is under active development. Functions, interfaces, database structures, graphics builds, folder structures, documents, and repository processes may change, be incomplete, or contain errors. Use is at the user's own responsibility.

3. Permitted use

DevBox may be used to locally organize, document, develop, and maintain software projects owned by the user or projects the user is authorized to work on. Users are

responsible for ensuring that they have the necessary rights for processed content, data, source code, images, font files, repositories, and external services.

4. External programs and services

DevBox may start or cooperate with locally installed programs such as Inkscape, GIMP, Git, Git Credential Manager, and installers after explicit user action. The respective providers or rights holders are responsible for those programs, their licenses, updates, data processing, and terms of use.

A repository push, graphics build, or product-specific process is prepared or started only after explicit user action. The user is responsible for target paths, repository URL, branch, visibility, commit content, and authorization to publish.

5. No warranty

To the extent permitted by law, DevBox is provided without any assurance of availability, error-free operation, fitness for a particular purpose, or complete compatibility. In particular, DevBox does not replace backups, legal review, security review, professional acceptance, or release approval.

6. Protection of the development state

DevBox uses temporary workspaces for export, graphics, and publication operations. Nevertheless, the user remains responsible for creating current backups of the project state and checking target locations, database changes, cleanup operations, Git operations, and tool starts before they are performed.

7. Documents and legal content

README, license, privacy, or terms documents generated with DevBox may contain technical templates or draft versions. Before public or commercial use, they must be reviewed and, where necessary, adapted to the specific product, its actual data flows, distribution channels, contractual relationships, and applicable law.

8. Limitation of liability

To the extent permitted by law, the author is not liable for indirect damage, data loss, lost benefits, misconfiguration, incorrect publications, faulty asset builds, or consequences of using external programs and services. Mandatory legal claims remain unaffected.

9. Changes

This draft version may be adapted as DevBox evolves.

3. License Reference

License: Zero-Clause BSD License 0BSD

The complete license terms are provided in the accompanying license file.

4. Project-Specific Notes

Purpose: DevBox is intended to turn recurring and error-prone development work into traceable local tools. This includes maintaining product and manufacturer data, structured documentation, prepared publication steps, integration of local creative applications, and controlled execution of product-specific development functions.

The application provides a shared working foundation for products of the CYXnTrol Development Platform. Instead of keeping workflows only as memory, folder conventions, or a collection of console commands, they are retained as data, scripts, GUI functions, and verifiable process chains.

For deskNode, DevBox additionally serves as a development and configuration interface for product versions, UX themes, device categories, consumer symbols, and prepared graphics packages. The later deskNode runtime is intended to receive reproducible data and pre-rendered assets from this process.

Context: The CYXnTrol Development Platform emerged from practical development work involving many small tools, data states, graphics files, documents, and test workflows. As the number of products and functions grows, it is no longer sufficient to keep information only in individual files or in memory. A stable project root, traceable data sources, repeatable temporary workspaces, and clearly separated product modules are required.

DevBox is the response to this need. It provides a local development hub in which manufacturer and product data, documentation, global structure rules, external tools, repository preparation, and specialized product functions are brought together.

deskNode is the first product integrated as an active DevBox area. Its role is to manage and visualize smart plugs and their connected consumers. For those consumers, not only names but also categories, stable technical IDs, PNG sources, theme data, and pre-rendered graphical states are placed into a reproducible workflow. This removes the need to manually create symbol variants for every theme and state combination.

Repository Note: The DevBox repository represents the CYXnTrol Development Platform itself, not a finished end-user product. It serves as a development record, a transparent work sample, and a foundation for maintaining the platform.

The published state is not copied directly from the productive project root. DevBox creates a separate cleaned root_dir state. It contains intended source code, documents, and assets, while local runtime data, temporary remnants, installers, unnecessary build output, and unpublished data remain outside the publication package.

The temporary publication root treats applications as a deliberate delivery window: the directory is cleaned first. It is then populated with applications/deskNode/resources/scripts and all contained files. __pycache__ directories as well as .pyc and .pyo files remain excluded. The real local product directory is not changed by this process.

DevBox is developed in an AI-assisted iterative workflow. Requirements, architecture, data models, UX decisions, test cases, and acceptance are directed by the project owner. Implementation is created with AI-assisted development tools and checked against defined functional requirements.

Copyright (c) 2026 Markus Walloner